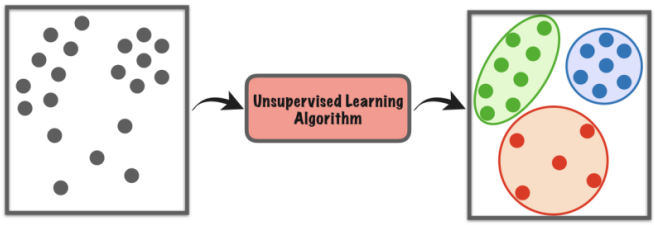
IPD440 - APRENDIZAJE DE MAQUINAS

Clase 3: Clustering

Prof. Nicolás Torres
nicolas.torresr@usm.cl
Ingeniería Civil Telemática
Departamento de Electrónica
Universidad Técnica Federico Santa María

¿Qué es clustering?

Formar grupos de datos que son "similares".



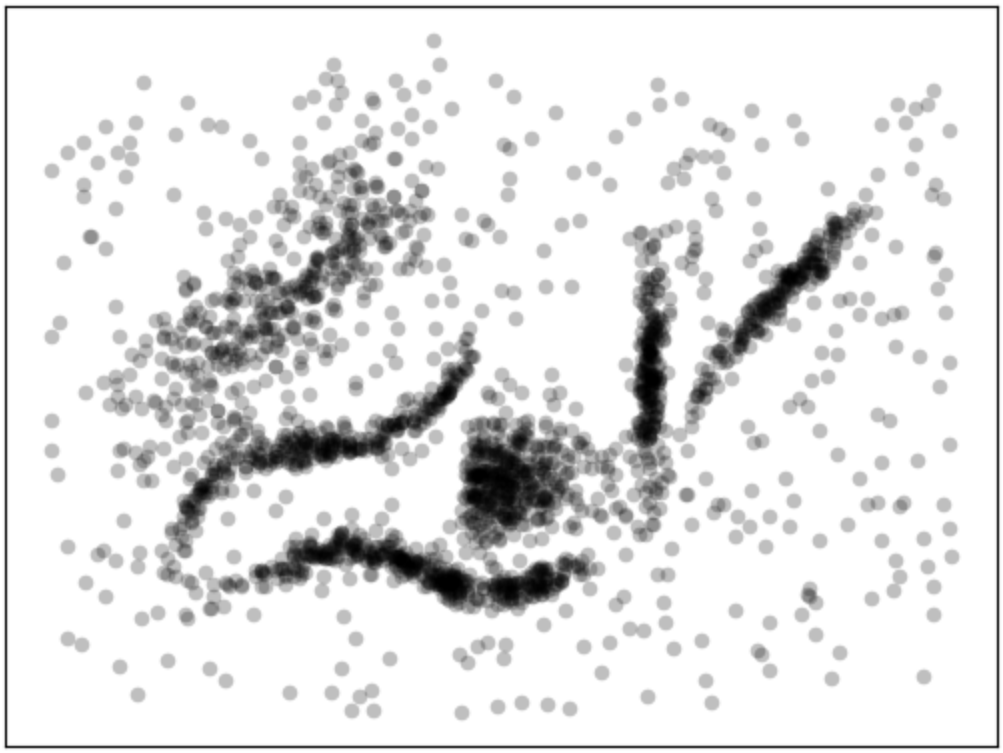
In [1]:
```
import matplotlib
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import sklearn.cluster as cluster
import sklearn.manifold as manifold
import time
from sklearn.datasets import make_blobs, make_circles, make_moons
```

In [2]:
```
import warnings
warnings.filterwarnings("ignore")
```

In [3]:
```
plot_kwds = {'alpha' : 0.25, 's' : 30, 'linewidths':0}
```

In [4]:
```
data = np.load('clusterable_data.npy')
```

In [5]:
```
plt.scatter(data.T[0], data.T[1], c='k', **plot_kwds)
frame = plt.gca()
frame.axes.get_xaxis().set_visible(False)
frame.axes.get_yaxis().set_visible(False)
```



K-means

El algoritmo KMeans agrupa los datos formando K grupos disjuntos de igual varianza, cada uno descrito por la media μ_j de los datos en el clúster (centroide).

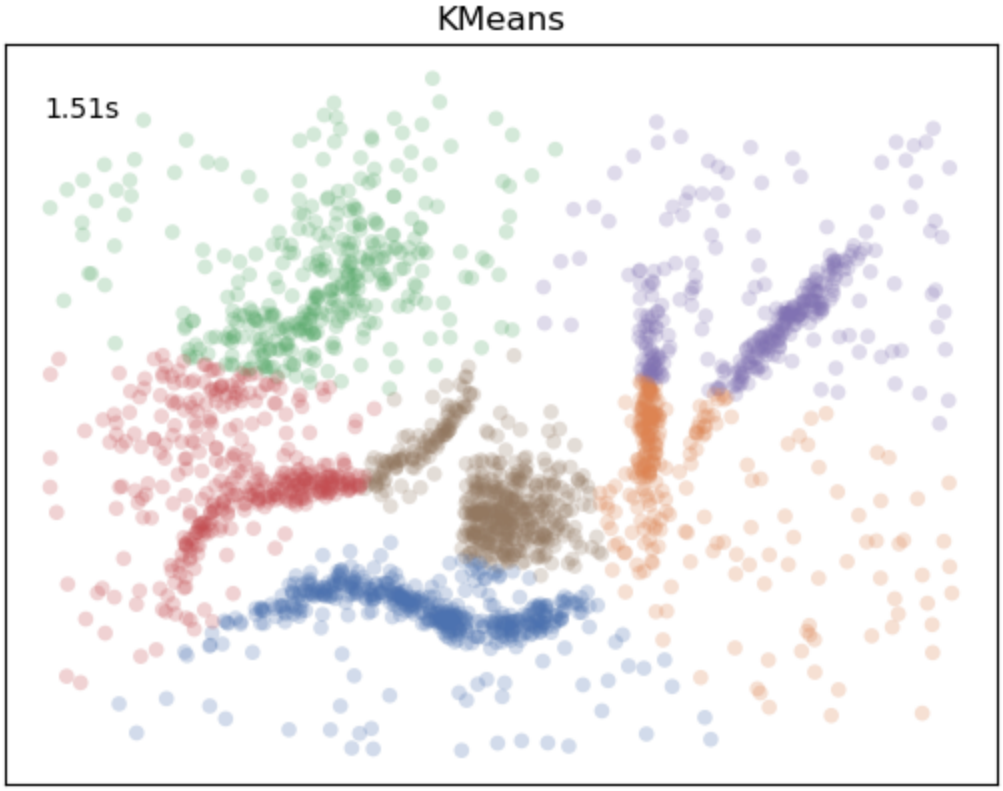
Minimiza un criterio conocido como Inercia que es la suma de la distancia al cuadrado dentro del grupo (WCSS o SSE).

$$\sum_{i=0}^{n} \min_{\mu_j \in C}(||x_i - \mu_j||^2)$$

Este algoritmo requiere que se especifique el número de clusters, siempre converge y escala bien a un gran número de datos.

In [6]:
```
def plot_clusters(data, algorithm, args, kwds):
    start_time = time.time()
    labels = algorithm(*args, **kwds).fit_predict(data)
    end_time = time.time()
    palette = sns.color_palette('deep', np.unique(labels).max() + 1)
    colors = [palette[x] if x >= 0 else (0.0, 0.0, 0.0) for x in labels]
    plt.scatter(data.T[0], data.T[1], c=colors, **plot_kwds)
    frame = plt.gca()
    frame.axes.get_xaxis().set_visible(False)
    frame.axes.get_yaxis().set_visible(False)
    plt.title('{}'.format(str(algorithm.__name__)))
    plt.text(-0.5, 0.6, '{:.2f}s'.format(end_time - start_time))
```

In [7]:
```
plot_clusters(data, cluster.KMeans, (), {'n_clusters':6})
```



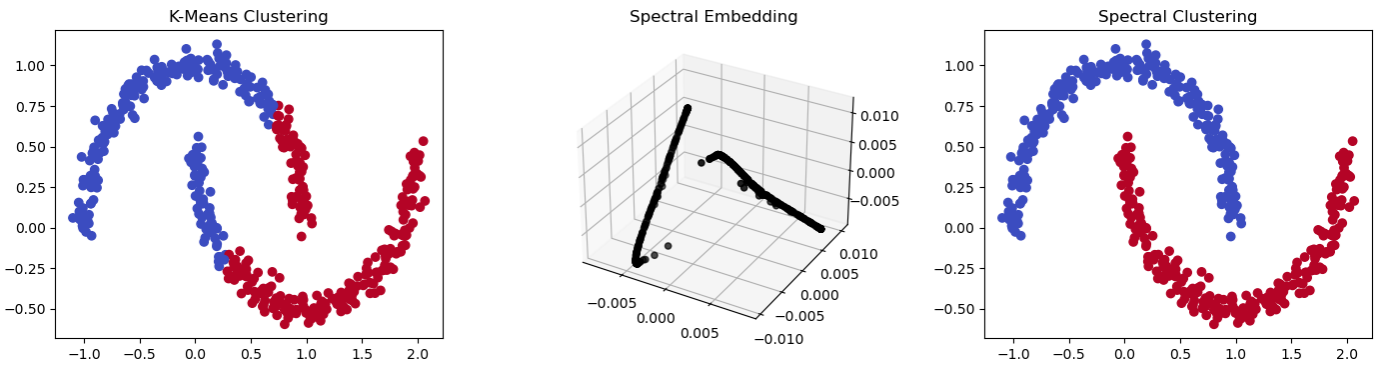
Spectral clustering

SpectralClustering calcula un embedding usando la matriz de afinidad de los datos y luego sobre esa representación realiza clustering usando, generalmente, KMeans.

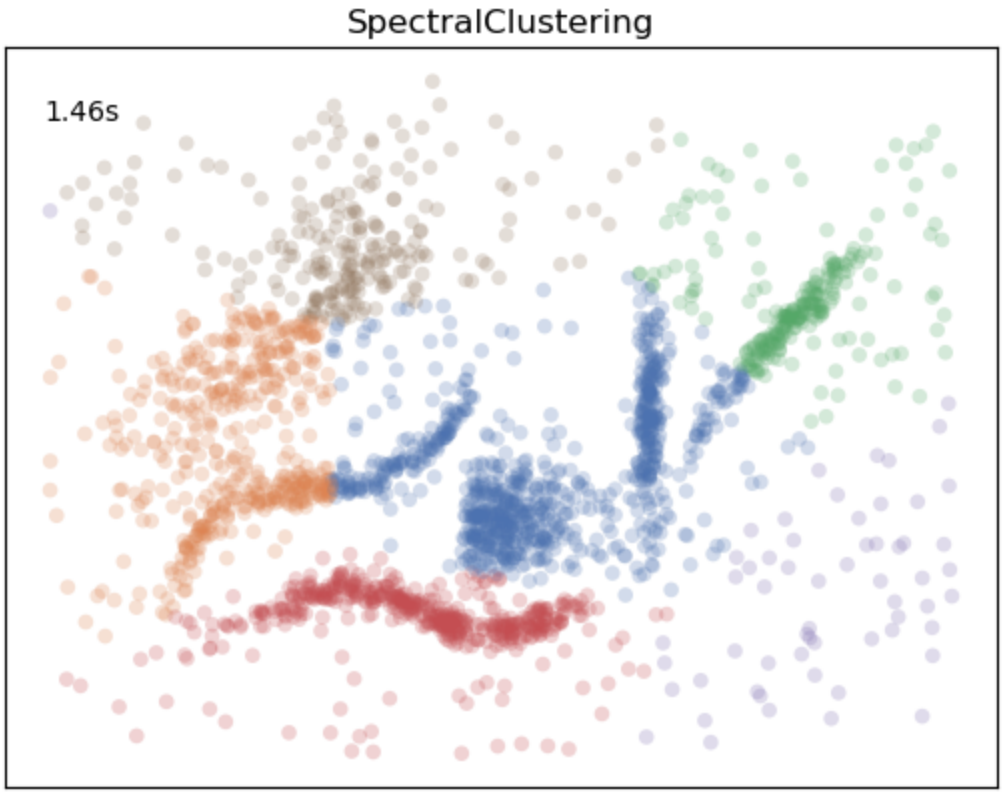
In [8]:
```
def plot_example_spectral(data):
    kmeans = cluster.KMeans(n_clusters=2).fit(data)
    embedding = manifold.SpectralEmbedding(n_components=3, affinity='nearest_neighbors').fit_tra
    spectral = cluster.SpectralClustering(n_clusters=2, affinity="nearest_neighbors").fit(data)
    fig = plt.figure(figsize=(17, 4))
    ax1 = fig.add_subplot(131)
    ax2 = fig.add_subplot(132, projection='3d')
    ax3 = fig.add_subplot(133)
    ax1.scatter(data[:,0], data[:,1], c=kmeans.labels_, cmap=plt.cm.coolwarm)
    ax1.set_title("K-Means Clustering")
    ax2.scatter(embedding[:,0], embedding[:,1], embedding[:,2], c='k')
    ax2.set_title("Spectral Embedding")
    ax3.scatter(data[:,0], data[:,1], c=spectral.labels_, cmap=plt.cm.coolwarm)
    ax3.set_title("Spectral Clustering")
    plt.show()
```

In [9]:
```
n_samples = 500
moons,_ = make_moons(n_samples=n_samples, noise=0.05)
```

In [10]:
```
plot_example_spectral(moons)
```



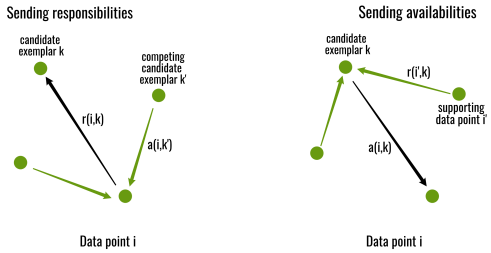
In [11]:
```
plot_clusters(data, cluster.SpectralClustering, (), {'n_clusters':6})
```



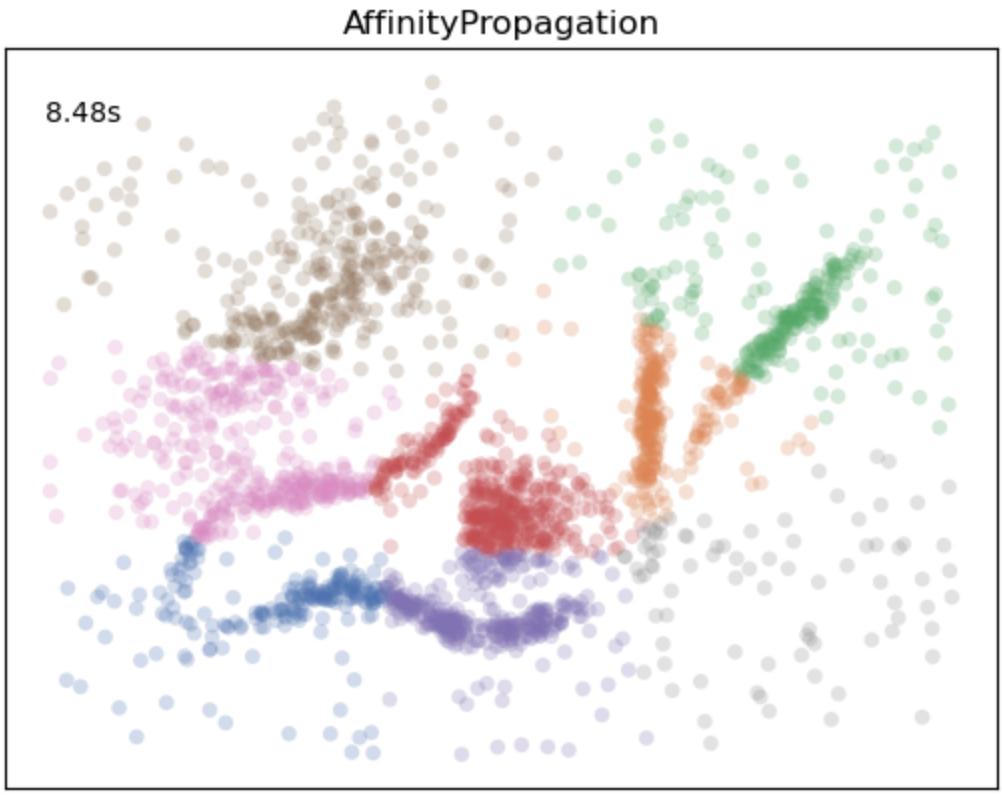
Affinity Propagation

Forma clústers enviando mensajes de "responsabilidad" y "disponibilidad" iterativamente entre pares de datos hasta la convergencia y los grupos quedan representados por ejemplares que recibieron alta demanda de solicitudes afines.

Cada punto de los datos envía mensajes de solicitud a los otros informando sus deseos de unirse y cada objetivo responde informando su disponibilidad de asociación dado el atractivo del mensaje. Una vez que el remitente se asocia con uno de sus solicitantes, se convierte en el ejemplar del dato (centroide). Todos los puntos con el mismo ejemplar se agrupan en el mismo clúster.



In [12]:
```
plot_clusters(data, cluster.AffinityPropagation, (), {'preference':-5.0, 'damping':0.95})
```



Hierarchical clustering

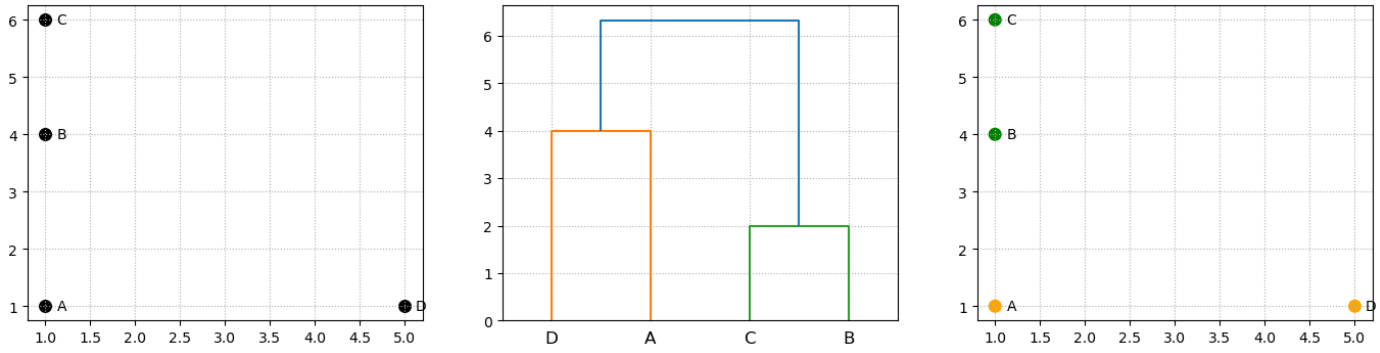
El agrupamiento jerárquico crea clústers anidados fusionándolos o dividiéndolos sucesivamente. Esta jerarquía de grupos se representa como un árbol (o dendrograma). La raíz del árbol es el único clúster que reúne todas las muestras, siendo las hojas los grupos con una sola muestra.

In [13]:
```
from scipy.cluster.hierarchy import dendrogram, linkage
```

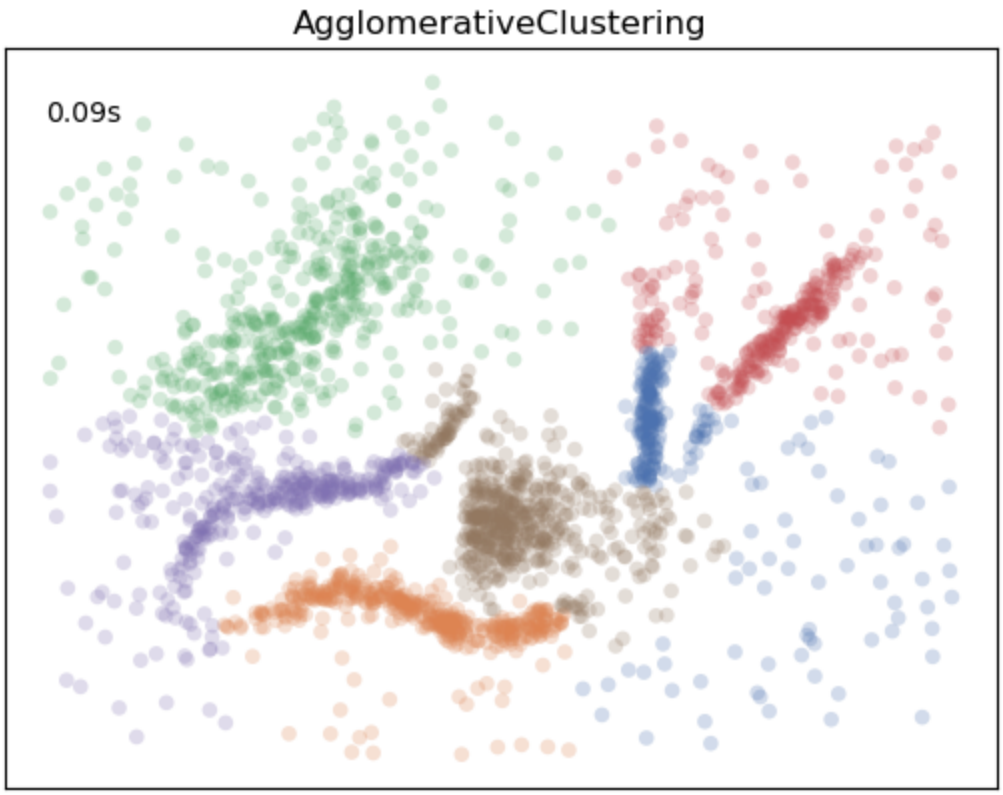
In [14]:
```
X = np.array([[1, 1], [1, 4], [1, 6], [5, 1]])
```

In [15]:
```
def plot_example_hac(data, link, k):
    hac = cluster.AgglomerativeClustering(n_clusters=k, linkage=link).fit(data)
    colors = ['orange','green','blue','red']
    fig = plt.figure(figsize=(17, 4))
    ax1 = fig.add_subplot(131)
    ax2 = fig.add_subplot(132)
    ax3 = fig.add_subplot(133)
    ax1.scatter(data[:, 0], data[:, 1], c='black', s=70)
    for label, x, y in zip(["A","B","C","D"], data[:, 0], data[:, 1]):
        ax1.annotate(label,xy=(x, y), xytext=(15,-5),textcoords='offset points', ha='right', va=
    ax1.grid(linestyle="dotted")
    dendrogram(linkage(data, link),labels=["A","B","C","D"],distance_sort='descending',ax=ax2)
    ax2.grid(linestyle="dotted")
    ax3.scatter(data[:, 0], data[:, 1], c=[colors[l_] for l_ in hac.labels_], s=70)
    for label, x, y in zip(["A","B","C","D"], data[:, 0], data[:, 1]):
        ax3.annotate(label,xy=(x, y), xytext=(15,-5),textcoords='offset points', ha='right', va=
    ax3.grid(linestyle="dotted")
    plt.show()
```

In [16]:
```
plot_example_hac(X, 'ward', 2)
```



In [17]:
```
plot_clusters(data, cluster.AgglomerativeClustering, (), {'n_clusters':6, 'linkage':'ward'})
```



DBSCAN

Asume que los clústers se originan por áreas de alta densidad separadas por zonas de baja densidad que representan el ruido. Por ende, los grupos pueden tener cualquier forma. El componente central de cada grupo es el *core* (datos centrales en áreas de alta densidad). El parámetro `eps` controla el tamaño del área y `MinPts` la cantidad mínima de datos para considerar el área como un clúster.

In [18]:
```
X_sample, _ = make_blobs(n_samples=10, centers=3, n_features=2, random_state=0)
```

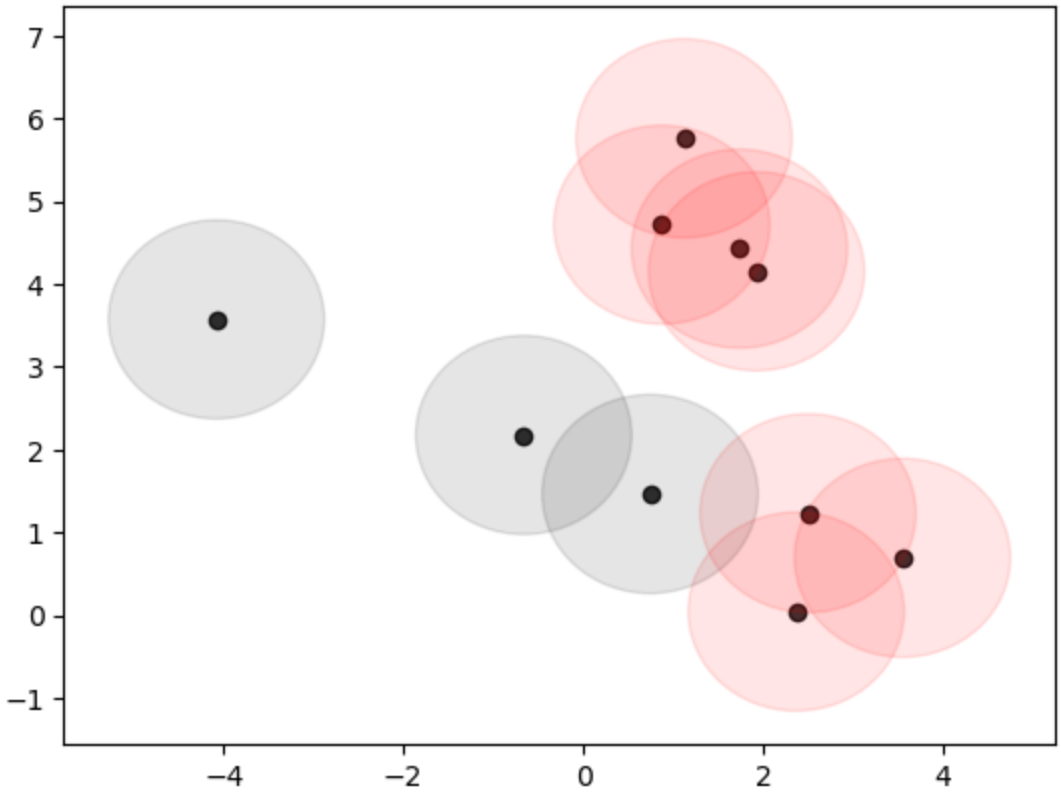
In [19]:
```
moons, _ = make_moons(n_samples=50, noise=0.05)
blobs, _ = make_blobs(n_samples=50, centers=[(-0.75,2.25), (1.0, 2.0)], cluster_std=0.25)
test_data = np.vstack([moons, blobs])
```

In [20]:
```
from sklearn.neighbors import NearestNeighbors
```

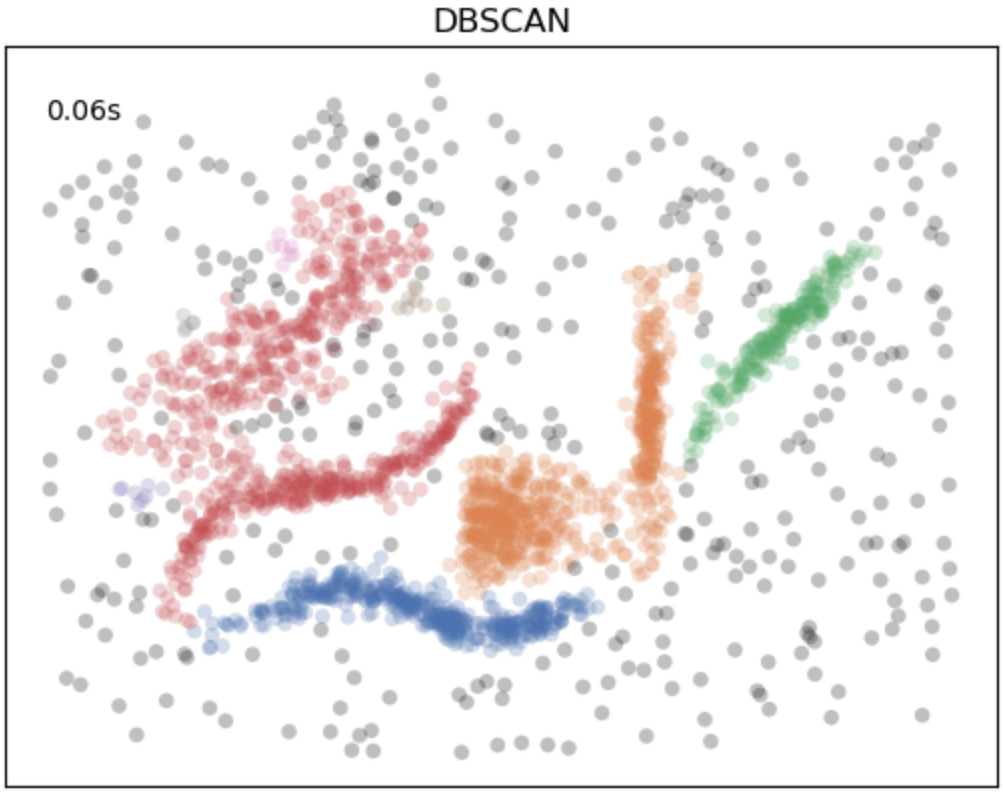
In [21]:
```
def plot_example_dbscan(data, eps=0.1, min_points=2):
    figure, axes = plt.subplots()
    plt.scatter(data[:,0], data[:,1], c='k', alpha=0.8)
    knn = NearestNeighbors(n_neighbors=min_points)
```

```
    knn.fit(data)
    for x,y in data:
        neigh_dist, neigh_ind = knn.kneighbors([(x,y)], min_points)
        if max(neigh_dist[0]) <= eps:
            cl = 'r'
        else:
            cl = 'k'
        c = plt.Circle((x,y), eps, color=cl, fill = True, alpha=0.1)
        plt.gca().add_patch(c)
```

In [22]:
```
plot_example_dbscan(X_sample, eps=1.2, min_points=2)
```



In [23]:
```
plot_clusters(data, cluster.DBSCAN, (), {'eps':0.025})
```



¿Qué hace que un clustering sea bueno?

Si se conocen las etiquetas reales de los datos, es posible usar métricas que miden la entropía (desorden) de cada clúster:

- **Homogeneidad:** cada grupo contiene solo miembros de una sola clase.
- **Integridad:** todos los miembros de una clase determinada se asignan al mismo grupo.
- **Pureza:** número total de objetos que se clasificaron correctamente.

Silhouette Coefficient

Si no se conocen las etiquetas reales, la evaluación debe realizarse utilizando el propio modelo. El Coeficiente Silueta se calcula para cada muestra y se compone de dos puntuaciones:

- a: La distancia media entre una muestra y todos los demás puntos del mismo clúster.
- b: La distancia media entre una muestra y todos los demás puntos en el clúster más cercano.

$$s = \frac{b - a}{max(a, b)}$$

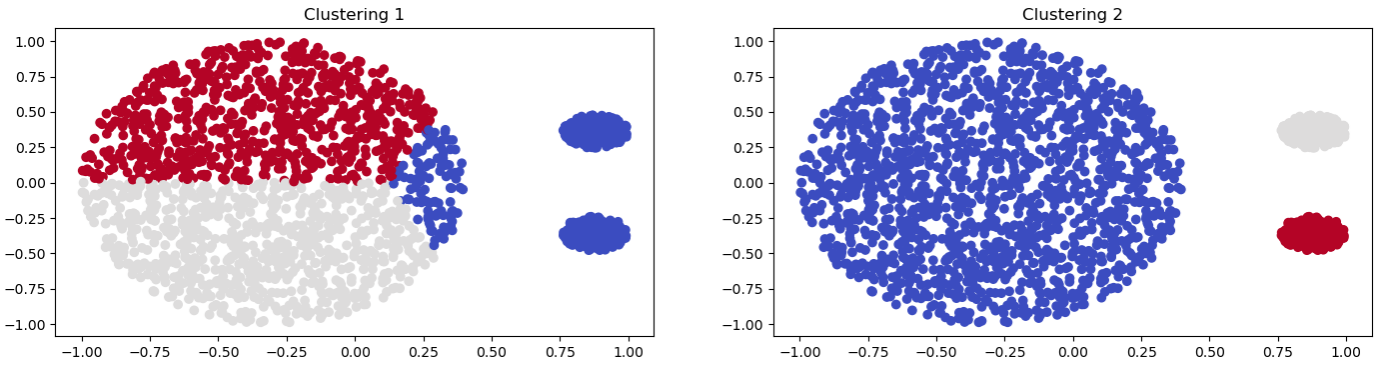
El mejor valor es 1 y el peor valor es -1 (datos asignados al clúster equivocado, ya que el otro grupo es más similar).

In [24]:
```
from numpy import genfromtxt
```

In [25]:
```
datos = genfromtxt('cure.txt', delimiter='\t')
```

In [26]:
```
cl1 = cluster.KMeans(n_clusters=3).fit(datos)
cl2 = cluster.DBSCAN(eps=0.3).fit(datos)
```

In [27]:
```
fig = plt.figure(figsize=(17, 4))
ax1 = fig.add_subplot(121)
ax2 = fig.add_subplot(122)
ax1.scatter(datos[:,0], datos[:,1], c=cl1.labels_, cmap=plt.cm.coolwarm)
ax1.set_title("Clustering 1")
ax2.scatter(datos[:,0], datos[:,1], c=cl2.labels_, cmap=plt.cm.coolwarm)
ax2.set_title("Clustering 2")
plt.show()
```



In [28]:
```
from sklearn.metrics import silhouette_score
```

In [29]:
```
print("Silhouette Score in Clustering 1:",round(silhouette_score(datos, cl1.labels_),2))
print("Silhouette Score in Clustering 2:",round(silhouette_score(datos, cl2.labels_),2))
```

```
Silhouette Score in Clustering 1: 0.44
Silhouette Score in Clustering 2: 0.42
```

El Clustering 1 tiene mejor Coeficiente Silueta y menor Inercia o Suma de los Cuadrados Dentro del Grupo (WCSS). Por ende, sería una mejor agrupación. Sin embargo, el Clustering 2 tiene más sentido al ojo humano.

¿Qué es un clúster?

Debemos preguntarnos, que es un clúster para nosotros y que buscamos formar.

Una noción avanzada sería formar componentes conexas de diferentes formas considerando distintos niveles de densidad.

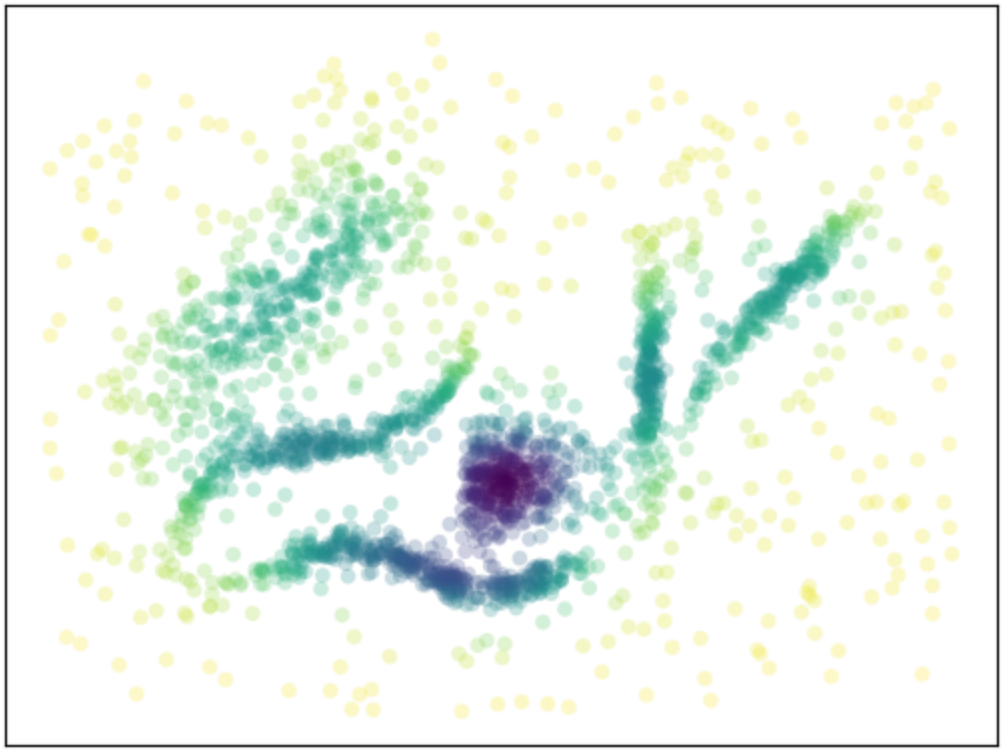
PDF

La estimación de densidad de Kernel (KDE) es una forma no paramétrica de estimar la función de densidad de probabilidad (PDF) de una variable aleatoria.

In [30]:
```
from scipy.stats import gaussian_kde
```

In [31]:
```
def plot_kde(data):
    # Calculate the point density
    xy = np.vstack([data.T[0], data.T[1]])
    z = gaussian_kde(xy)(xy)
    plt.scatter(data.T[0], data.T[1], c=z, **plot_kwds, cmap=plt.cm.viridis_r)
    frame = plt.gca()
    frame.axes.get_xaxis().set_visible(False)
    frame.axes.get_yaxis().set_visible(False)
```

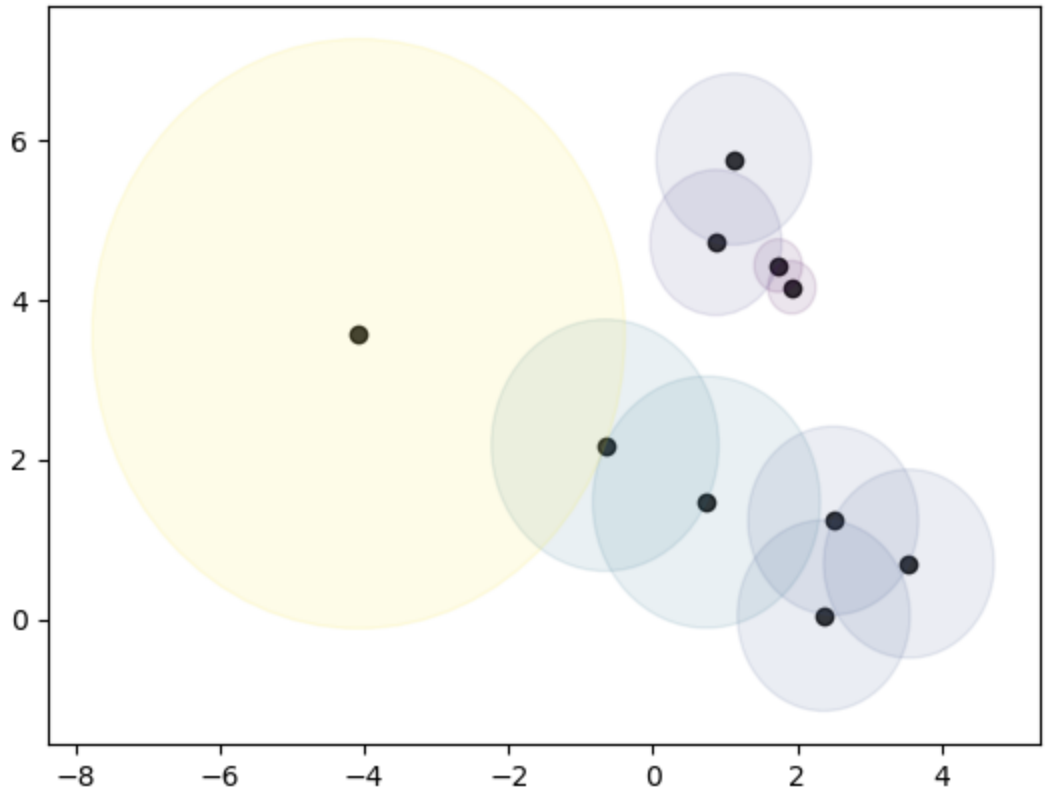
In [32]:
```
plot_kde(data)
```



DBSCAN con densidad variable

In [33]:
```
def plot_example_variable_density(data, min_cluster_size=2):
    figure, axes = plt.subplots()
    plt.scatter(data[:,0], data[:,1], c='k', alpha=0.8)
    knn = NearestNeighbors(n_neighbors=min_cluster_size)
    knn.fit(data)
    lst = []
    for x,y in data:
        neigh_dist, neigh_ind = knn.kneighbors([(x,y)], min_cluster_size)
        lst.append(max(neigh_dist[0]))
    minima = min(lst)
    maxima = max(lst)
    norm = matplotlib.colors.Normalize(vmin=minima, vmax=maxima, clip=True)
    mapper = plt.cm.ScalarMappable(norm=norm, cmap=plt.cm.viridis)
    for i,(x,y) in enumerate(data):
        d = mapper.to_rgba(lst[i])
        c = plt.Circle((x,y), lst[i], color=d, fill = True, alpha=0.1)
        plt.gca().add_patch(c)
```

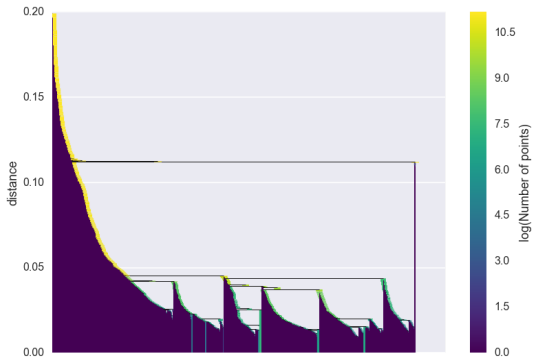
In [34]:
```
plot_example_variable_density(X_sample, min_cluster_size=2)
```



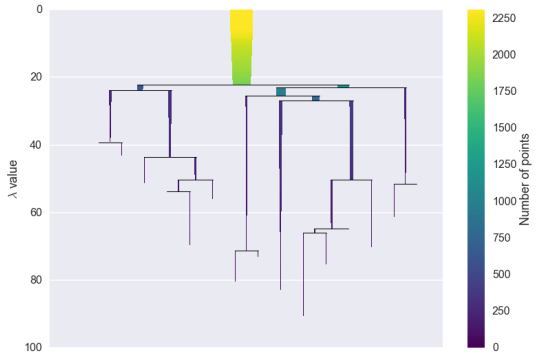
HDBSCAN

Mejora DBSCAN siendo capaz de encontrar diferentes escalas de densidad posibles mediante la construcción de una representación alternativa a través de un agrupamiento jerárquico. Puede ser visto como realizar DBSCAN sobre todos los posibles valores de `eps` y quedarse con los mejores clústers en cada pasada, por ende, es invariante a la escala.

Density Cluster Tree



Simplified Cluster Tree



In [35]:
```
# HBSCAN está en la versión 1.3 de scikit-learn
# pip install --upgrade scikit-learn
```

In [36]:
```
import sklearn
print('The scikit-learn version is {}.'.format(sklearn.__version__))
```

```
The scikit-learn version is 1.3.0.
```

In [37]:
```
plot_clusters(data, cluster.HDBSCAN, (), {'min_cluster_size':15})
```

